



Recurrent Neural Networks

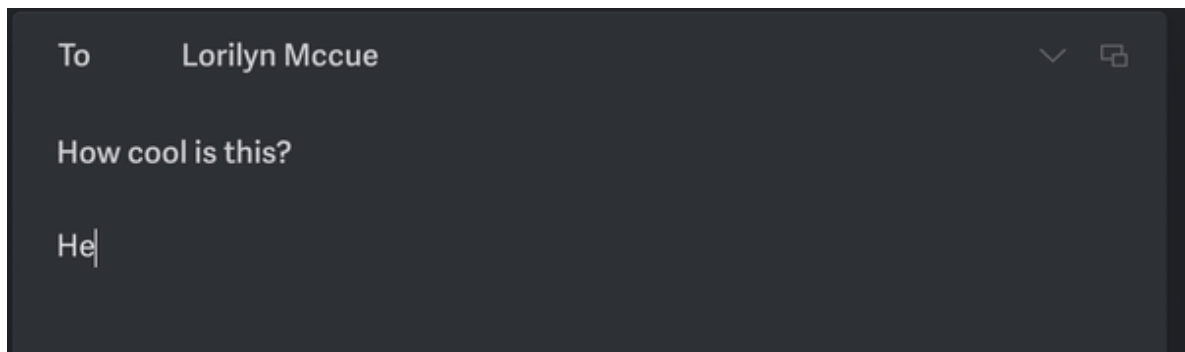
Inclass Project 3 - MA4144

This project contains multiple tasks to be completed, some require written answers. Questions that required written answers are given in blue fonts. Almost all written questions are open ended, they do not have a correct or wrong answer. You are free to give your opinions, but please provide related answers within the context.

After finishing project run the entire notebook once and **save the notebook as a pdf** (File menu -> Save and Export Notebook As -> PDF). You are **required to upload both this ipynb file and the PDF on moodle**.

Outline of the project

The aim of the project is to build a RNN model to suggest autocompletion of half typed words. You may have seen this in many day today applications; typing an email, a text message etc. For example, suppose you type in the four letter "univ", the application may suggest you to autocomplete it by "university".



We will train a RNN to suggest possible autocompletes given 3\$ - 4\$ starting letters. That is if we input a string "univ" hopefully we expect to see an output like "university", "universal" etc.

For this we will use a text file (wordlist.txt) containing 10,000 common English words (you'll find the file on the moodle link). The list of words will be the "**vocabulary**" for our model.

We will use the Python **torch library** to implement our autocomplete model.

Use the below cell to use any include any imports

```
In [2]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
```

Section 1: Preparing the vocabulary

```
In [3]: WORD_SIZE = 13
```

Q1. In the following cell provide code to load the text file (each word is in a newline), then extract the words (in lowercase) into a list.

For practical reasons of training the model we will only use words that are longer than 3 letters and that have a maximum length of WORD_SIZE (this will be a constant we set at the beginning - you can change this and experiment with different WORD_SIZES). As seen above it is set to 13.

So out of the extracted list of words filter out those words that match our criteria on word length.

To train our model it is convenient to have words/strings of equal length. We will choose to convert every word to length of WORD_SIZE, by adding underscores to the end of the word if it is initially shorter than WORD_SIZE. For example, we will convert the word "university" (word length 10) into "university__" (wordlength 13). In your code include this conversion as well.

Store the processed WORD_SIZE lengthed strings in a list called vocab.

```
In [4]: # Load words from file
with open('wordlist.txt', 'r', encoding='utf-8') as f:
    words = [line.strip().lower() for line in f.readlines()]

# Define helper functions for word processing
def is_valid_length(term):
    return 3 < len(term) <= WORD_SIZE

def pad_word(term):
    return term + '_' * (WORD_SIZE - len(term))

# Process vocabulary: filter by length then apply padding
valid_words = []
for word in words:
    if is_valid_length(word):
```

```
        valid_words.append(word)

vocab = []
for word in valid_words:
    padded_word = pad_word(word)
    vocab.append(padded_word)

# Display results
print(f"Loaded {len(words)} words")
print(f"Vocabulary after filtering/padding: {len(vocab)}")
vocab
```

Loaded 10000 words

Vocabulary after filtering/padding: 8847

```
Out[4]: ['aaron_____',
'abandoned_____',
'aberdeeen_____',
'abilities_____',
'ability_____',
'able_____',
'aboriginal_____',
'abortion_____',
'about_____',
'above_____',
'abraham_____',
'abroad_____',
'absence_____',
'absent_____',
'absolute_____',
'absolutely_____',
'absorption_____',
'abstract_____',
'abstracts_____',
'abuse_____',
'academic_____',
'academics_____',
'academy_____',
'accent_____',
'accept_____',
'acceptable_____',
'acceptance_____',
'accepted_____',
'accepting_____',
'accepts_____',
'access_____',
'accessed_____',
'accessibility',
'accessible_____',
'accessing_____',
'accessories_____',
'accessory_____',
'accident_____',
'accidents_____',
'accommodate_____',
'accommodation',
'accompanied_____',
'accompanying_____',
'accomplish_____',
'accomplished_____',
'accordance_____',
'according_____',
'accordingly_____',
'account_____',
'accounting_____',
'accounts_____',
'accreditation',
'accredited_____',
'accuracy_____']
```

'accurate____',
'accurately____',
'accused____',
'acdbentity____',
'acer____',
'achieve____',
'achieved____',
'achievement____',
'achievements____',
'achieving____',
'acid____',
'acids____',
'acknowledge____',
'acknowledged____',
'acne____',
'acoustic____',
'acquire____',
'acquired____',
'acquisition____',
'acquisitions____',
'acre____',
'acres____',
'acrobat____',
'across____',
'acrylic____',
'acting____',
'action____',
'actions____',
'activated____',
'activation____',
'active____',
'actively____',
'activists____',
'activities____',
'activity____',
'actor____',
'actors____',
'actress____',
'acts____',
'actual____',
'actually____',
'acute____',
'adam____',
'adams____',
'adaptation____',
'adapted____',
'adapter____',
'adapters____',
'adaptive____',
'adaptor____',
'added____',
'addiction____',
'adding____',
'addition____',

'additional____',
'additionally_',
'additions____',
'address____',
'addressed____',
'addresses____',
'addressing____',
'adds____',
'adelaide____',
'adequate____',
'adidas____',
'adipex____',
'adjacent____',
'adjust____',
'adjustable____',
'adjusted____',
'adjustment____',
'adjustments____',
'admin____',
'administered_',
'administrator',
'admission____',
'admissions____',
'admit____',
'admitted____',
'adobe____',
'adolescent____',
'adopt____',
'adopted____',
'adoption____',
'adrian____',
'adsl____',
'adult____',
'adults____',
'advance____',
'advanced____',
'advancement____',
'advances____',
'advantage____',
'advantages____',
'adventure____',
'adventures____',
'adverse____',
'advert____',
'advertise____',
'advertisement',
'advertiser____',
'advertisers____',
'advertising____',
'advice____',
'advise____',
'advised____',
'advisor____',
'advisors____',

'advisory____',
'advocacy____',
'advocate____',
'adware____',
'aerial____',
'aerospace____',
'affair____',
'affairs____',
'affect____',
'affected____',
'affecting____',
'affects____',
'affiliate____',
'affiliated____',
'affiliates____',
'affiliation____',
'afford____',
'affordable____',
'afghanistan____',
'afraid____',
'africa____',
'african____',
'after____',
'afternoon____',
'afterwards____',
'again____',
'against____',
'aged____',
'agencies____',
'agency____',
'agenda____',
'agent____',
'agents____',
'ages____',
'aggregate____',
'aggressive____',
'aging____',
'agree____',
'agreed____',
'agreement____',
'agreements____',
'agrees____',
'agricultural____',
'agriculture____',
'ahead____',
'aids____',
'aimed____',
'aims____',
'aircraft____',
'airfare____',
'airline____',
'airlines____',
'airplane____',
'airport____',

'airports____',
'alabama____',
'alan____',
'alarm____',
'alaska____',
'albania____',
'albany____',
'albert____',
'alberta____',
'album____',
'albums____',
'albuquerque__',
'alcohol____',
'alert____',
'alerts____',
'alex____',
'alexander____',
'alexandria____',
'alfred____',
'algebra____',
'algeria____',
'algorithm____',
'algorithms____',
'alias____',
'alice____',
'alien____',
'align____',
'alignment____',
'alike____',
'alive____',
'allah____',
'allan____',
'alleged____',
'allen____',
'allergy____',
'alliance____',
'allied____',
'allocated____',
'allocation____',
'allow____',
'allowance____',
'allowed____',
'allowing____',
'allows____',
'alloy____',
'almost____',
'alone____',
'along____',
'alot____',
'alpha____',
'alphabetical_',
'alpine____',
'already____',
'also____',

'alter_____',
'altered_____',
'alternate_____',
'alternative_____',
'alternatively',
'alternatives_____',
'although_____',
'alto_____',
'aluminium_____',
'aluminum_____',
'alumni_____',
'always_____',
'amanda_____',
'amateur_____',
'amazing_____',
'amazon_____',
'amazoncom_____',
'amazoncouk_____',
'ambassador_____',
'amber_____',
'ambien_____',
'ambient_____',
'amend_____',
'amended_____',
'amendment_____',
'amendments_____',
'amenities_____',
'america_____',
'american_____',
'americans_____',
'americas_____',
'amino_____',
'among_____',
'amongst_____',
'amount_____',
'amounts_____',
'ampland_____',
'amplifier_____',
'amsterdam_____',
'anaheim_____',
'anal_____',
'analog_____',
'analyses_____',
'analysis_____',
'analyst_____',
'analysts_____',
'analytical_____',
'analyze_____',
'analyzed_____',
'anatomy_____',
'anchor_____',
'ancient_____',
'andale_____',
'anderson_____',

'andorra_____',
'andrea_____',
'andreas_____',
'andrew_____',
'andrews_____',
'andy_____',
'angel_____',
'angela_____',
'angeles_____',
'angels_____',
'anger_____',
'angle_____',
'angola_____',
'angry_____',
'animal_____',
'animals_____',
'animated_____',
'animation_____',
'anime_____',
'anna_____',
'anne_____',
'annex_____',
'annie_____',
'anniversary_____',
'annotated_____',
'annotation_____',
'announce_____',
'announced_____',
'announcement_____',
'announcements_____',
'announces_____',
'annoying_____',
'annual_____',
'annually_____',
'anonymous_____',
'another_____',
'answer_____',
'answered_____',
'answering_____',
'answers_____',
'antarctica_____',
'antenna_____',
'anthony_____',
'anthropology_____',
'anti_____',
'antibodies_____',
'antibody_____',
'anticipated_____',
'antigua_____',
'antique_____',
'antiques_____',
'antivirus_____',
'antonio_____',
'anxiety_____',

'anybody_____',
'anymore_____',
'anyone_____',
'anything_____',
'anytime_____',
'anyway_____',
'anywhere_____',
'apache_____',
'apart_____',
'apartment_____',
'apartments_____',
'apnic_____',
'apollo_____',
'apparatus_____',
'apparel_____',
'apparent_____',
'apparently_____',
'appeal_____',
'appeals_____',
'appear_____',
'appearance_____',
'appeared_____',
'appearing_____',
'appears_____',
'appendix_____',
'apple_____',
'appliance_____',
'appliances_____',
'applicable_____',
'applicant_____',
'applicants_____',
'application_____',
'applications_____',
'applied_____',
'applies_____',
'apply_____',
'applying_____',
'appointed_____',
'appointment_____',
'appointments_____',
'appraisal_____',
'appreciate_____',
'appreciated_____',
'appreciation_____',
'approach_____',
'approaches_____',
'appropriate_____',
'approval_____',
'approve_____',
'approved_____',
'approx_____',
'approximate_____',
'approximately_____',
'apps_____'

'april_____',
'aqua_____',
'aquarium_____',
'aquatic_____',
'arab_____',
'arabia_____',
'arabic_____',
'arbitrary_____',
'arbitration_____',
'arcade_____',
'arch_____',
'architect_____',
'architects_____',
'architectural_____',
'architecture_____',
'archive_____',
'archived_____',
'archives_____',
'arctic_____',
'area_____',
'areas_____',
'arena_____',
'argentina_____',
'argue_____',
'argued_____',
'argument_____',
'arguments_____',
'arise_____',
'arising_____',
'arizona_____',
'arkansas_____',
'arlington_____',
'armed_____',
'armenia_____',
'armor_____',
'arms_____',
'armstrong_____',
'army_____',
'arnold_____',
'around_____',
'arrange_____',
'arranged_____',
'arrangement_____',
'arrangements_____',
'array_____',
'arrest_____',
'arrested_____',
'arrival_____',
'arrivals_____',
'arrive_____',
'arrived_____',
'arrives_____',
'arrow_____',
'arthritis_____',

'arthur_____',
'article_____',
'articles_____',
'artificial_____',
'artist_____',
'artistic_____',
'artists_____',
'arts_____',
'artwork_____',
'aruba_____',
'asbestos_____',
'ascii_____',
'ashley_____',
'asia_____',
'asian_____',
'aside_____',
'asin_____',
'asked_____',
'asking_____',
'asks_____',
'aspect_____',
'aspects_____',
'aspnet_____',
'assault_____',
'assembled_____',
'assembly_____',
'assess_____',
'assessed_____',
'assessing_____',
'assessment_____',
'assessments_____',
'asset_____',
'assets_____',
'assign_____',
'assigned_____',
'assignment_____',
'assignments_____',
'assist_____',
'assistance_____',
'assistant_____',
'assisted_____',
'assists_____',
'associate_____',
'associated_____',
'associates_____',
'association_____',
'associations_____',
'assume_____',
'assumed_____',
'assumes_____',
'assuming_____',
'assumption_____',
'assumptions_____',
'assurance_____',

'assure_____',
'assured_____',
'asthma_____',
'astrology_____',
'astronomy_____',
'asus_____',
'athens_____',
'athletes_____',
'athletic_____',
'athletics_____',
'atlanta_____',
'atlantic_____',
'atlas_____',
'atmosphere_____',
'atmospheric_____',
'atom_____',
'atomic_____',
'attach_____',
'attached_____',
'attachment_____',
'attachments_____',
'attack_____',
'attacked_____',
'attacks_____',
'attempt_____',
'attempted_____',
'attempting_____',
'attempts_____',
'attend_____',
'attendance_____',
'attended_____',
'attending_____',
'attention_____',
'attitude_____',
'attitudes_____',
'attorney_____',
'attorneys_____',
'attract_____',
'attraction_____',
'attractions_____',
'attractive_____',
'attribute_____',
'attributes_____',
'auburn_____',
'auckland_____',
'auction_____',
'auctions_____',
'audi_____',
'audience_____',
'audio_____',
'audit_____',
'auditor_____',
'august_____',
'aurora_____',

'austin_____',
'australia_____',
'australian_____',
'austria_____',
'authentic_____',
'author_____',
'authorities_____',
'authority_____',
'authorization_____',
'authorized_____',
'authors_____',
'auto_____',
'automated_____',
'automatic_____',
'automatically_____',
'automation_____',
'automobile_____',
'automobiles_____',
'automotive_____',
'autos_____',
'autumn_____',
'availability_____',
'available_____',
'avatar_____',
'avenue_____',
'average_____',
'aviation_____',
'avoid_____',
'avoiding_____',
'avon_____',
'award_____',
'awarded_____',
'awards_____',
'aware_____',
'awareness_____',
'away_____',
'awesome_____',
'awful_____',
'axis_____',
'azerbaijan_____',
'babe_____',
'babes_____',
'babies_____',
'baby_____',
'bachelor_____',
'back_____',
'backed_____',
'background_____',
'backgrounds_____',
'backing_____',
'backup_____',
'bacon_____',
'bacteria_____',
'bacterial_____',

'badge_____',
'badly_____',
'baghdad_____',
'bags_____',
'bahamas_____',
'bahrain_____',
'bailey_____',
'baker_____',
'baking_____',
'balance_____',
'balanced_____',
'bald_____',
'bali_____',
'ball_____',
'ballet_____',
'balloon_____',
'ballot_____',
'balls_____',
'baltimore_____',
'banana_____',
'band_____',
'bands_____',
'bandwidth_____',
'bang_____',
'bangbus_____',
'bangkok_____',
'bangladesh_____',
'bank_____',
'banking_____',
'bankruptcy_____',
'banks_____',
'banned_____',
'banner_____',
'banners_____',
'baptist_____',
'barbados_____',
'barbara_____',
'barbie_____',
'barcelona_____',
'bare_____',
'barely_____',
'bargain_____',
'bargains_____',
'barn_____',
'barnes_____',
'barrel_____',
'barrier_____',
'barriers_____',
'barry_____',
'bars_____',
'base_____',
'baseball_____',
'based_____',
'baseline_____',

'basement____',
'basename____',
'bases____',
'basic____',
'basically____',
'basics____',
'basin____',
'basis____',
'basket____',
'basketball____',
'baskets____',
'bass____',
'batch____',
'bath____',
'bathroom____',
'bathrooms____',
'baths____',
'batman____',
'batteries____',
'battery____',
'battle____',
'battlefield____',
'bdsm____',
'beach____',
'beaches____',
'beads____',
'beam____',
'bean____',
'beans____',
'bear____',
'bearing____',
'bears____',
'beast____',
'beastality____',
'beastiality____',
'beat____',
'beatles____',
'beats____',
'beautiful____',
'beautifully____',
'beauty____',
'beaver____',
'became____',
'because____',
'become____',
'becomes____',
'becoming____',
'bedding____',
'bedford____',
'bedroom____',
'bedrooms____',
'beds____',
'beef____',
'been____',

'beer_____',
'before_____',
'began_____',
'begin_____',
'beginner_____',
'beginners_____',
'beginning_____',
'begins_____',
'begun_____',
'behalf_____',
'behavior_____',
'behavioral_____',
'behaviour_____',
'behind_____',
'beijing_____',
'being_____',
'beings_____',
'belarus_____',
'belfast_____',
'belgium_____',
'belief_____',
'beliefs_____',
'believe_____',
'believed_____',
'believes_____',
'belize_____',
'belkin_____',
'bell_____',
'belle_____',
'belly_____',
'belong_____',
'belongs_____',
'below_____',
'belt_____',
'belts_____',
'bench_____',
'benchmark_____',
'bend_____',
'beneath_____',
'beneficial_____',
'benefit_____',
'benefits_____',
'benjamin_____',
'bennett_____',
'benz_____',
'berkeley_____',
'berlin_____',
'bermuda_____',
'bernard_____',
'berry_____',
'beside_____',
'besides_____',
'best_____',
'bestiality_____'

'bestsellers__',
'beta____',
'beth____',
'better____',
'betting____',
'betty____',
'between____',
'beverage____',
'beverages____',
'beverly____',
'beyond____',
'bhutan____',
'bias____',
'bible____',
'biblical____',
'bibliographic',
'bibliography_',
'bicycle____',
'bidder____',
'bidding____',
'bids____',
'bigger____',
'biggest____',
'bike____',
'bikes____',
'bikini____',
'bill____',
'billing____',
'billion____',
'bills____',
'billy____',
'binary____',
'bind____',
'binding____',
'bingo____',
'biodiversity_',
'biographies__',
'biography____',
'biol____',
'biological____',
'biology____',
'bios____',
'biotechnology',
'bird____',
'birds____',
'birmingham__',
'birth____',
'birthday____',
'bishop____',
'bitch____',
'bite____',
'bits____',
'bizarre____',
'bizrate____',

'black_____',
'blackberry____',
'blackjack____',
'blacks_____',
'blade_____',
'blades_____',
'blah_____',
'blair_____',
'blake_____',
'blame_____',
'blank_____',
'blanket_____',
'blast_____',
'bleeding_____',
'blend_____',
'bless_____',
'blessed_____',
'blind_____',
'blink_____',
'block_____',
'blocked_____',
'blocking_____',
'blocks_____',
'blog_____',
'blogger_____',
'bloggers_____',
'blogging_____',
'blogs_____',
'blond_____',
'blonde_____',
'blood_____',
'bloody_____',
'bloom_____',
'bloomberg_____',
'blow_____',
'blowing_____',
'blowjob_____',
'blowjobs_____',
'blue_____',
'blues_____',
'bluetooth_____',
'blvd_____',
'board_____',
'boards_____',
'boat_____',
'boating_____',
'boats_____',
'bobby_____',
'bodies_____',
'body_____',
'bold_____',
'bolivia_____',
'bolt_____',
'bomb_____',

'bond_____',
'bondage_____',
'bonds_____',
'bone_____',
'bones_____',
'bonus_____',
'boob_____',
'boobs_____',
'book_____',
'booking_____',
'bookings_____',
'bookmark_____',
'bookmarks_____',
'books_____',
'bookstore_____',
'bool_____',
'boolean_____',
'boom_____',
'boost_____',
'boot_____',
'booth_____',
'boots_____',
'booty_____',
'border_____',
'borders_____',
'bored_____',
'boring_____',
'born_____',
'borough_____',
'bosnia_____',
'boss_____',
'boston_____',
'both_____',
'bother_____',
'botswana_____',
'bottle_____',
'bottles_____',
'bottom_____',
'bought_____',
'boulder_____',
'boulevard_____',
'bound_____',
'boundaries_____',
'boundary_____',
'bouquet_____',
'boutique_____',
'bowl_____',
'bowling_____',
'boxed_____',
'boxes_____',
'boxing_____',
'boys_____',
'bracelet_____',
'bracelets_____',

```
'bracket_____',  
'brad_____',  
'bradford_____',  
'bradley_____',  
'brain_____',  
'brake_____',  
'brakes_____',  
'branch_____',  
'branches_____',  
'brand_____',  
'brandon_____',  
'brands_____',  
'bras_____',  
'brass_____',  
'brave_____',  
'brazil_____',  
'brazilian_____',  
'breach_____',  
'bread_____',  
'break_____',  
'breakdown_____',  
'breakfast_____',  
'breaking_____',  
'breaks_____',  
'breast_____',  
'breasts_____',  
'breath_____',  
'breathing_____',  
...]
```

In the above explanation it was mentioned "for practical reasons of training the model we will only use words that are longer than 3 letters and that have a certain maximum length". In your opinion what could be those practicals? Will it help to build a better model?

Answer

Reasons for Filtering Words by Length:

1. Reduce Noise:

- Short words (like "a" or "I") don't carry much semantic meaning.
- Long words are rare and complex, which can add unnecessary noise.

2. Improve Efficiency:

- Smaller vocabulary leads to faster training and reduced memory usage.

3. Enhance Generalization:

- Focusing on medium-length words prevents overfitting to rare words and allows the model to learn more generalizable patterns.

4. Control Vocabulary Size:

- Ensures the model doesn't handle an unmanageable vocabulary, improving model performance.

Filtering words by length **helps build a better model** by improving efficiency, generalization, and reducing complexity.

Q2 To input words into the model, we will need to convert each letter/character into a number. as we have seen above, the only characters in our list vocab will be the underscore and lowercase english letters. so we will convert these 27 characters into numbers as follows: underscore -> 0, 'a' -> 1, 'b' -> 2, ..., 'z' -> 26. In the following cell,

(i) Implement a method called `char_to_num`, that takes in a valid character and outputs its numerical assignment.

(ii) Implement a method called `num_to_char`, that takes in a valid number from 0 to 26 and outputs the corresponding character.

(iii) Implement a method called `word_to_numlist`, that takes in a word from our vocabulary and outputs a (torch) tensor of numbers that corresponds to each character in the word in that order. For example: the word "united_" will be converted to `tensor([21, 14, 9, 20, 5, 4, 0, 0, 0, 0, 0, 0])`. You are encouraged to use your `char_to_num` method for this.

(iv) Implement a method called `numlist_to_word`, that does the opposite of the above described `word_to_numlist`, given a tensor of numbers from 0 to 26, outputs the corresponding word. You are encouraged to use your `num_to_char` method for this.

Note: As mentioned since we are using the torch library we will be using tensors instead of the usual python lists or numpy arrays. Tensors are the list equivalent in torch. Torch models only accept tensors as input and they output tensors.

```
In [5]: def char_to_num(char):  
        if char == '_':  
            num = 0  
        elif 'a' <= char <= 'z':
```

```

        num = ord(char) - ord('a') + 1
    else:
        raise ValueError(f"Invalid character: {char}")

    return(num)

def num_to_char(num):
    if num == 0:
        char = '_'
    elif 1 <= num <= 26:
        char = chr(num - 1 + ord('a'))
    else:
        raise ValueError(f"Invalid number: {num}")
    return(char)

def word_to_numlist(word):
    numlist = torch.tensor(list(map(char_to_num, word)), dtype=torch.long)
    return(numlist)

def numlist_to_word(numlist):
    word = ''.join(map(lambda x: num_to_char(x.item()), numlist))
    return(word)

def vocab_to_tensor(vocab_list):
    """Convert entire vocabulary to tensor format efficiently"""
    return torch.stack([word_to_numlist(word) for word in vocab_list])

vocab_tensors = vocab_to_tensor(vocab)
print(f"Vocabulary converted to tensor format: {vocab_tensors.shape}")

```

Vocabulary converted to tensor format: torch.Size([8847, 13])

We convert letter into just numbers based on their alphabetical order, I claim that it is a very bad way to encode data such as letters to be fed into learning models, please write your explanation to or against my claim. If you are searching for reasons, the keyword 'categorical data' may be useful. Although the letters in our case are not treated as categorical data, the same reasons as for categorical data is applicable. Even if my claim is valid, at the end it won't matter due to something called "embedding layers" that we will use in our model. What is an embedding layer? What is it's purpose? Explain. (write answers in the cell below as a string)

In [14]: *#Write answer here as strings:*

```

answer_ = """
The claim is valid - encoding letters based on alphabetical order is indeed a
1. Artificial Ordering: Assigning 'a'=1, 'b'=2, etc. implies that 'b' is s
2. Distance Misrepresentation: The model might interpret that 'a' and 'c'
3. Categorical Nature: Letters are categorical data, not ordinal. There's

```


However, this issue is resolved by **embedding layers**. An embedding layer is

****Purpose of Embedding Layers:****

- Transform sparse, one-hot encoded inputs into dense vectors
- Learn meaningful relationships between characters during training
- Allow the model to discover that certain characters frequently appear together
- Convert categorical data into a continuous space where mathematical operations can be performed

The embedding layer essentially learns a better representation than our naive
"""

Section 2: Implementing the Autocomplete model

We will implement a RNN LSTM model. The [video tutorial](#) will be useful. Our model will be only one hidden layer, but feel free to sophisticate with more layers after the project for your own experiments.

Our model will contain all the training and prediction methods as single package in a class (autocompleteModel) we will define and implement below.

```
In [7]: LEARNING_RATE = 0.005
```

```
In [8]: class autocompleteModel(nn.Module):

    #Constructor
    def __init__(self, alphabet_size, embed_dim, hidden_size, num_layers):
        super().__init__()

        #Set the input parameters to self parameters
        self.alphabet_size = alphabet_size
        self.embed_dim = embed_dim
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        #Initialize the layers in the model:
        #1 embedding layer, 1 - LSTM cell (hidden layer), 1 fully connected layer
        self.embedding = nn.Embedding(alphabet_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, alphabet_size)

        #Feedforward
        def forward(self, character, hidden_state, cell_state):

            #Perform feedforward in order
            #1. Embed the input (one character represented by a number)
            #2. Feed the embedded output to the LSTM cell
            #3. Feed the LSTM output to the fully connected layer to obtain the output
            #4. return the output, and both the hidden state and cell state from the LSTM
```

```

        # Embed the input character - character should be a single value
        embedded = self.embedding(character.unsqueeze(0).unsqueeze(0)) # Shape: [1, 1, embedding_dim]

        # Feed to LSTM cell
        lstm_out, (hidden_state, cell_state) = self.lstm(embedded, (hidden_state, cell_state))

        # Feed to fully connected layer
        output = self.fc(lstm_out.squeeze(0).squeeze(0)) # Remove sequence and batch dimension

        return output, hidden_state, cell_state

#Initialize the first hidden state and cell state (for the start of a word)
def initial_state(self):

    h0 = torch.zeros(self.num_layers, 1, self.hidden_size)
    c0 = torch.zeros(self.num_layers, 1, self.hidden_size)

    return (h0, c0)

#Train the model in epochs given the vocab, the training will be fed in batches
def trainModel(self, vocab, epochs = 5, batch_size = 100):

    #Convert the model into train mode
    self.train()

    #Set the optimizer (ADAM), you may need to provide the model parameter
    optimizer = optim.Adam(self.parameters(), lr=LEARNING_RATE)
    criterion = nn.CrossEntropyLoss()

    #Keep a log of the loss at the end of each training cycle.
    loss_log = []

    # Convert vocab to tensors for efficient batch processing
    vocab_tensors = torch.stack([word_to_numlist(word) for word in vocab])
    dataset_size = len(vocab_tensors)
    num_iter = dataset_size // batch_size

    print(f"Starting training... {num_iter} batches per epoch")

    for e in range(epochs):
        epoch_loss = 0
        batch_count = 0

        #Shuffle the vocab list the start of each epoch
        indices = torch.randperm(dataset_size)
        vocab_shuffled = vocab_tensors[indices]

        for i in range(num_iter):

            # Get batch
            start_idx = i * batch_size
            end_idx = min(start_idx + batch_size, dataset_size)

```

```

        batch = vocab_shuffled[start_idx:end_idx]

        #Set the loss to zero, initialize the optimizer with zero_grad
        optimizer.zero_grad()

        # Process entire batch efficiently
        inputs = batch[:, :-1] # All chars except last: (batch_size,
        targets = batch[:, 1:] # All chars except first: (batch_size,

        # Forward pass for entire batch
        embedded = self.embedding(inputs) # (batch_size, seq_len-1, e
        lstm_out, _ = self.lstm(embedded) # (batch_size, seq_len-1, h
        outputs = self.fc(lstm_out) # (batch_size, seq_len-1, alphabe

        # Compute loss for entire batch
        loss = criterion(outputs.reshape(-1, self.alphabet_size), targ

        #Compute the average loss per word in the batch and perform ba
        loss.backward()

        #Update model parameters using the optimizer
        optimizer.step()

        #Update the loss_log and epoch stats
        loss_log.append(loss.item())
        epoch_loss += loss.item()
        batch_count += 1

        # Print epoch results with average loss
        avg_epoch_loss = epoch_loss / batch_count
        print(f"Epoch {e + 1}/{epochs}, Average Loss: {avg_epoch_loss:.4f}")

    #Plot a graph of the variation of the loss.
    plt.figure(figsize=(10, 6))
    plt.plot(loss_log)
    plt.xlabel('Batch')
    plt.ylabel('Loss')
    plt.title('Training Loss Over Time')
    plt.grid(True)
    plt.show()

#Perform autocomplete given a sample of strings (typically 3-5 starting let
def autocomplete(self, sample, temperature=1.0):

    #Convert the model into evaluation mode
    self.eval()
    completed_list = []

    #In the following loop for each sample item initialize hidden and cell
    #You will have to convert the output into a softmax (you may use your
    with torch.no_grad():
        for literal in sample:
            # Initialize states

```

```

hidden_state, cell_state = self.initial_state()

# Convert input to lowercase and get character numbers
literal_lower = literal.lower()
current_word = []

# Process existing characters
for char in literal_lower:
    char_num = torch.tensor(char_to_num(char))
    output, hidden_state, cell_state = self.forward(char_num,
    current_word.append(char)

# Generate remaining characters
for _ in range(WORD_SIZE - len(literal_lower)):
    # Convert output to probabilities using temperature
    probs = torch.softmax(output / temperature, dim=0)

    # Sample next character
    next_char_num = torch.multinomial(probs, 1).item()

    # Stop if underscore (padding)
    if next_char_num == 0:
        break

    # Add character to word and continue
    next_char = num_to_char(next_char_num)
    current_word.append(next_char)

    # Use this character as input for next iteration
    char_tensor = torch.tensor(next_char_num)
    output, hidden_state, cell_state = self.forward(char_tensor

# Convert to string and add to results
completed_word = ''.join(current_word)
completed_list.append(completed_word)

return completed_list

```

Section 3: Using and evaluating the model

(i) Initialize and train autoCompleteModels using different embedding dimensions and hidden layer sizes. Use different learning rates, epochs, batch sizes. Train the best model you can.

(ii) Evaluate it on different samples of partially filled in words to test your model. Eg: ["univ", "math", "neur", "engin"] etc.

(iii) Set your best model, to the variable best_model. This model will be tested against random inputs (3-4 starting strings of common English words). **This will be the main contributor for your score in this project.**

```
In [11]: # Define model hyperparameters
alphabet_size = 27
embed_dim = 64
hidden_size = 128
num_layers = 1
epochs = 60
batch_size = 128

# Display training configuration
print(f"Vocabulary size: {len(vocab)} words")
print(f"Training with {epochs} epochs, batch size {batch_size}")

# Create model instance with specified architecture
model = autoCompleteModel(
    alphabet_size=alphabet_size,
    embed_dim=embed_dim,
    hidden_size=hidden_size,
    num_layers=num_layers
)

# Calculate and display model complexity
total_params = sum(p.numel() for p in model.parameters())
print(f"Model parameters: {total_params:,}")

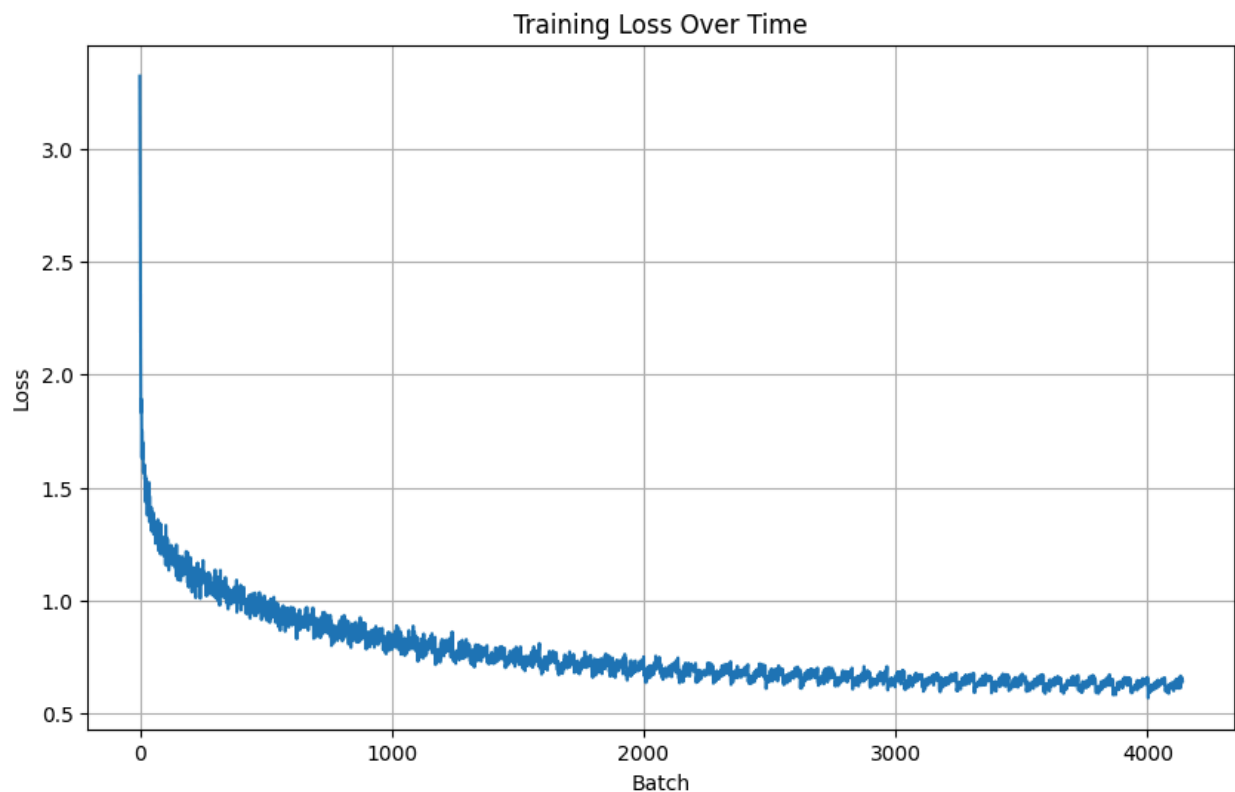
# Execute training process
model.trainModel(
    vocab=vocab,
    epochs=epochs,
    batch_size=batch_size
)

# Store the trained model as the best model
best_model = model
```

Vocabulary size: 8847 words
Training with 60 epochs, batch size 128
Model parameters: 104,539
Starting training... 69 batches per epoch

Epoch 1/60,	Average Loss: 1.5402
Epoch 2/60,	Average Loss: 1.2311
Epoch 3/60,	Average Loss: 1.1514
Epoch 4/60,	Average Loss: 1.0994
Epoch 5/60,	Average Loss: 1.0556
Epoch 6/60,	Average Loss: 1.0177
Epoch 7/60,	Average Loss: 0.9860
Epoch 8/60,	Average Loss: 0.9584
Epoch 9/60,	Average Loss: 0.9323
Epoch 10/60,	Average Loss: 0.9090
Epoch 11/60,	Average Loss: 0.8882
Epoch 12/60,	Average Loss: 0.8696
Epoch 13/60,	Average Loss: 0.8518
Epoch 14/60,	Average Loss: 0.8361
Epoch 15/60,	Average Loss: 0.8206
Epoch 16/60,	Average Loss: 0.8093
Epoch 17/60,	Average Loss: 0.7967
Epoch 18/60,	Average Loss: 0.7840
Epoch 19/60,	Average Loss: 0.7741
Epoch 20/60,	Average Loss: 0.7648
Epoch 21/60,	Average Loss: 0.7559
Epoch 22/60,	Average Loss: 0.7476
Epoch 23/60,	Average Loss: 0.7408
Epoch 24/60,	Average Loss: 0.7326
Epoch 25/60,	Average Loss: 0.7255
Epoch 26/60,	Average Loss: 0.7193
Epoch 27/60,	Average Loss: 0.7156
Epoch 28/60,	Average Loss: 0.7102
Epoch 29/60,	Average Loss: 0.7030
Epoch 30/60,	Average Loss: 0.6990
Epoch 31/60,	Average Loss: 0.6948
Epoch 32/60,	Average Loss: 0.6889
Epoch 33/60,	Average Loss: 0.6829
Epoch 34/60,	Average Loss: 0.6810
Epoch 35/60,	Average Loss: 0.6768
Epoch 36/60,	Average Loss: 0.6730
Epoch 37/60,	Average Loss: 0.6711
Epoch 38/60,	Average Loss: 0.6688
Epoch 39/60,	Average Loss: 0.6654
Epoch 40/60,	Average Loss: 0.6624
Epoch 41/60,	Average Loss: 0.6605
Epoch 42/60,	Average Loss: 0.6564
Epoch 43/60,	Average Loss: 0.6535
Epoch 44/60,	Average Loss: 0.6532
Epoch 45/60,	Average Loss: 0.6490
Epoch 46/60,	Average Loss: 0.6479
Epoch 47/60,	Average Loss: 0.6461
Epoch 48/60,	Average Loss: 0.6446
Epoch 49/60,	Average Loss: 0.6428
Epoch 50/60,	Average Loss: 0.6405

Epoch 51/60, Average Loss: 0.6390
 Epoch 52/60, Average Loss: 0.6356
 Epoch 53/60, Average Loss: 0.6340
 Epoch 54/60, Average Loss: 0.6335
 Epoch 55/60, Average Loss: 0.6311
 Epoch 56/60, Average Loss: 0.6295
 Epoch 57/60, Average Loss: 0.6290
 Epoch 58/60, Average Loss: 0.6280
 Epoch 59/60, Average Loss: 0.6268
 Epoch 60/60, Average Loss: 0.6247



```
In [13]: test_samples = [
    # Original samples
    "univ", "math", "neur", "engin", "comp", "prog", "scie", "tech",

    # Additional challenging samples
    "phys", "chem", "biol", "psych", "phil", "hist", "econ", "poli",
    "algo", "data", "mach", "arti", "soft", "hard", "netw", "syst",
    "analy", "stat", "calc", "geom", "trig", "alge", "numb", "prob",
    "theo", "prac", "expe", "rese", "stud", "proj", "work", "task",
    "solv", "opti", "effi", "perf", "test", "vali", "veri", "impl",

    # Complex and challenging examples
    "biochem", "quantum", "environ", "micro", "macro", "nano", "cyber",
    "neuro", "astro", "geo", "hydro", "electr", "mechan", "thermo",
    "crypto", "viral", "bacte", "molec", "atom", "proto", "synth",
    "struct", "dynam", "kinetic", "static", "elastic", "plastic", "graph",
    "matrix", "vector", "scalar", "tensor", "complex", "simple", "linear",
    "nonlin", "finite", "infin", "contin", "discr", "random", "determ",
    "optim", "maxim", "minim", "constrain", "unconstrain", "global", "local",
```

```
    "approx", "exact", "numeric", "symbol", "analog", "digit", "binary"
]

print(f"\nTesting autocompletion on {len(test_samples)} samples:")

results = best_model.autocomplete(test_samples, temperature=0.000001)
for sample, result in zip(test_samples, results):
    print(f"{sample} -> {result}")

print("\nModel ready for evaluation!")
```


Testing autocompletion on 104 samples:

univ -> universal
math -> mathematical
neur -> neural
engin -> engineer
comp -> complete
prog -> programmers
scie -> sciences
tech -> techniques
phys -> physio
chem -> chemical
biol -> biology
psych -> psychological
phil -> phillips
hist -> historic
econ -> economic
poli -> policy
algo -> algorithm
data -> databases
mach -> machine
arti -> artist
soft -> software
hard -> hard
netw -> networks
syst -> system
analy -> analysis
stat -> statutory
calc -> calculator
geom -> geometry
trig -> trigger
alge -> algeria
numb -> number
prob -> probability
theo -> theories
prac -> practices
expe -> experiments
rese -> reservoir
stud -> studios
proj -> projectors
work -> works
task -> task
solv -> solve
opti -> optimization
effi -> efficient
perf -> performer
test -> testing
vali -> validation
veri -> verification
impl -> implemented
biochem -> biochem
quantum -> quantum
environ -> environments
micro -> microsoft
macro -> macro

nano -> nano
cyber -> cyber
neuro -> neurous
astro -> astronomy
geo -> george
hydro -> hydrogen
electr -> electronic
mechan -> mechanically
thermo -> thermo
crypto -> crypton
viral -> viral
bacte -> bacteria
molec -> molecules
atom -> atomic
proto -> protocols
synth -> synthetical
struct -> structures
dynam -> dynamics
kinetic -> kinetical
static -> static
elastic -> elastical
plastic -> plastics
graph -> graphic
matrix -> matrix
vector -> vector
scalar -> scalar
tensor -> tensor
complex -> complex
simple -> simple
linear -> linear
nonlin -> nonlines
finite -> finite
infin -> infinite
contin -> continues
discr -> discrete
random -> random
determ -> determines
optim -> optimization
maxim -> maximum
minim -> minimal
constrain -> constraint
unconstrain -> unconstrain
global -> global
local -> local
approx -> approximate
exact -> exact
numeric -> numeric
symbol -> symbol
analog -> analog
digit -> digit
binary -> binary

Model ready for evaluation!